

COTEPECOS

COLEGIO TÉCNICO PROFESIONAL

Sistema de Notas Estudiantiles

I Etapa — Diseño de un Sistema de Notas (POO básica)

Proyecto de Programación · Fundamentos de Programación Orientada a Objetos en Java

Estudiante: Damian Quirós Jiménez

Sección: 11-5

Subárea: Programación Web

Nivel: 11°

Profesor(a): _____

Fecha de entrega: 14 y 15 de octubre, 2026

Índice

1. **Introducción**
2. **Diseño de las clases principales del sistema**
3. **Atributos y métodos de cada clase**
4. **Encapsulamiento: uso de atributos privados y métodos get / set**
5. **Constructores de cada clase**
6. **El método main: qué pasa cuando se crea un objeto**
7. **Diagrama de clases (UML)**
8. **Diagrama de casos de uso (UML)**
9. **Diagrama de secuencia (UML)**
10. **Conclusión**

Introducción

Cualquier colegio necesita llevar un control de los estudiantes, los cursos que reciben y las notas que van sacando. Ese proceso, que normalmente se hace a mano o en una hoja de cálculo, es exactamente el tipo de problema que la Programación Orientada a Objetos (POO) resuelve muy bien: en vez de tener datos sueltos por todos lados, se organizan en **clases**, que son como "moldes" a partir de los cuales se crean **objetos** con su propia información.

En este documento se diseña, de forma teórica, un **Sistema de Notas Estudiantiles** en Java. Todavía no se ejecuta nada en NetBeans (eso corresponde a etapas posteriores del proyecto); en esta primera etapa lo que se evalúa es que se entienda *cómo se estructura* un programa orientado a objetos: qué clases tiene, qué atributos y métodos lleva cada una, cómo se protegen los datos con encapsulamiento, cómo se construyen los objetos y qué ocurre realmente cuando el método `main` los pone a funcionar. Al final se incluyen los tres diagramas UML que representan gráficamente todo lo anterior.

El sistema se pensó con cuatro clases: **Estudiante**, **Curso**, **Nota** y **Profesor**. Las tres primeras son las que pide el documento como mínimo; se agregó `Profesor` porque tiene sentido dentro de un sistema de notas real (alguien tiene que impartir el curso y registrar la nota) y porque, más adelante en el proyecto, esa clase se necesita para trabajar herencia.

1. Diseño de las clases principales del sistema

Indicador 1

Antes de escribir código hay que decidir qué "cosas" del mundo real va a representar el programa. En este sistema aparecen cuatro elementos que se repiten todo el tiempo cuando se habla de notas en un colegio, y cada uno se convirtió en una clase:

Clase	¿Qué representa dentro del sistema?
Estudiante	La persona que está matriculada, recibe cursos y acumula notas. Es el centro del sistema: casi todo gira alrededor de calcular y mostrar su información.
Curso	La materia que se imparte (por ejemplo Matemática u Programación Web). Agrupa estudiantes y está a cargo de un profesor.
Nota	El resultado numérico de una evaluación concreta (examen, proyecto, quiz), asociado a un estudiante y a un curso.
Profesor	La persona encargada de impartir uno o varios cursos y registrar las notas de sus estudiantes.

Estas cuatro clases no están sueltas: se relacionan entre sí (un profesor imparte cursos, un curso tiene estudiantes matriculados, un estudiante acumula notas, un curso también lleva el registro de las notas que se generan en él). Esas relaciones, con su multiplicidad, se representan más adelante en el diagrama de clases (punto 6).

2. Atributos y métodos de cada clase

Indicador 2

Cada clase se definió con al menos 3 atributos (la información que guarda cada objeto) y al menos 2 métodos (las acciones que ese objeto puede hacer). Los atributos son `private` —se explica el porqué en el punto 4— y los métodos son `public`, que es la forma en que el resto del programa puede "hablarle" al objeto.

Clase Estudiante

Atributo	Tipo	Descripción
identificacion	String	Número de identificación del estudiante.
nombre	String	Nombre completo del estudiante.
promedio	double	Promedio actual del estudiante.
correo	String	Correo electrónico de contacto.

Método	Descripción
mostrarDatos(): void	Imprime en pantalla la información principal del estudiante.
estaAprobado(): boolean	Indica si el promedio actual es igual o mayor a 70.

Clase Curso

Atributo	Tipo	Descripción
codigo	String	Código que identifica al curso (ej. "PRW-11B").
nombre	String	Nombre del curso.
profesorEncargado	String	Nombre del profesor que imparte el curso.
cantidadEstudiantes	int	Cantidad de estudiantes matriculados.

Método	Descripción
mostrarInformacion(): void	Imprime en pantalla los datos del curso.
agregarEstudiante(): void	Aumenta en uno la cantidad de estudiantes matriculados.

Clase Nota

Atributo	Tipo	Descripción
valor	double	Valor numérico de la nota (0 a 100).
tipoEvaluacion	String	Tipo de evaluación (examen, proyecto, quiz, etc.).
fecha	String	Fecha en la que se registró la nota.

Método	Descripción
mostrarNota(): void	Imprime en pantalla el detalle de la nota.
esAprobatoria(): boolean	Indica si esa nota específica es igual o mayor a 70.

Clase Profesor

Atributo	Tipo	Descripción
identificacion	String	Número de identificación del profesor.
nombre	String	Nombre completo del profesor.
especialidad	String	Materia en la que se especializa.

Método	Descripción
mostrarDatos(): void	Imprime en pantalla la información del profesor.
asignarCurso(c: Curso): void	Indica que el profesor quedó a cargo de un curso.

3. Encapsulamiento: atributos privados y métodos get / set

Indicador 3

Encapsular significa que nadie desde afuera de la clase puede tocar directamente los atributos: todos se declaran `private`. Si algo quiere leer o cambiar un dato, tiene que pasar obligatoriamente por un método `get` (para leer) o `set` (para escribir). Esto se hace por una razón concreta: si un atributo fuera público, cualquier parte del programa podría, por ejemplo, poner el promedio de un estudiante en `-40` o en `9999`, y nadie lo estaría controlando. Con encapsulamiento, ese control queda centralizado adentro de la clase.

Así se ve aplicado en la clase `Estudiante` (el mismo patrón se repite en `Curso`, `Nota` y `Profesor`):

```
public class Estudiante {
    private String identificacion;
    private String nombre;
    private double promedio;
    private String correo;

    // métodos get y set: la única puerta de entrada a los atributos
    public String getIdentificacion() { return identificacion; }
    public void setIdentificacion(String id) { this.identificacion = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public double getPromedio() { return promedio; }
    public void setPromedio(double promedio) {
        // aquí se podría validar el rango antes de guardar el dato
        this.promedio = promedio;
    }

    public String getCorreo() { return correo; }
    public void setCorreo(String correo) { this.correo = correo; }
}
```

En palabras simples: el atributo `promedio` está "escondido" dentro del objeto. Si en `main` se quiere cambiar, no se hace `e1.promedio = 105` (eso ni siquiera compila porque es privado); se tiene que hacer `e1.setPromedio(88.5)`. Esto deja la puerta abierta a que, el día de mañana, dentro de `setPromedio` se

agregue una validación como "si el valor es mayor a 100, no lo guardes", sin que el resto del programa tenga que cambiar nada.

4. Constructores de cada clase

Indicador 4

El constructor es un método especial que tiene el mismo nombre que la clase, no tiene tipo de retorno, y se ejecuta automáticamente cada vez que se usa la palabra `new`. Su trabajo es inicializar los atributos principales para que el objeto nazca en un estado válido, en vez de nacer "vacío". Abajo se muestran las cuatro clases completas, ya con sus atributos privados, su constructor y sus métodos:

Clase Estudiante

```
public class Estudiante {
    private String identificacion;
    private String nombre;
    private double promedio;
    private String correo;

    public Estudiante(String identificacion, String nombre, String correo) {
        this.identificacion = identificacion;
        this.nombre = nombre;
        this.correo = correo;
        this.promedio = 0.0; // todo estudiante nuevo inicia sin promedio
    }

    public String getIdentificacion() { return identificacion; }
    public void setIdentificacion(String v) { identificacion = v; }
    public String getNombre() { return nombre; }
    public void setNombre(String v) { nombre = v; }
    public double getPromedio() { return promedio; }
    public void setPromedio(double v) { promedio = v; }
    public String getCorreo() { return correo; }
    public void setCorreo(String v) { correo = v; }

    public void mostrarDatos() {
        System.out.println("Estudiante: " + nombre + " (" + identificacion + ") "
            + " - Promedio: " + promedio);
    }

    public boolean estaAprobado() {
        return promedio >= 70;
    }
}
```

Clase Curso

```
public class Curso {
    private String codigo;
    private String nombre;
    private String profesorEncargado;
    private int cantidadEstudiantes;

    public Curso(String codigo, String nombre, String profesorEncargado) {
        this.codigo = codigo;
        this.nombre = nombre;
    }
}
```

```

        this.profesorEncargado = profesorEncargado;
        this.cantidadEstudiantes = 0;
    }

    public String getCodigo() { return codigo; }
    public void setCodigo(String v) { codigo = v; }
    public String getNombre() { return nombre; }
    public void setNombre(String v) { nombre = v; }
    public String getProfesorEncargado() { return profesorEncargado; }
    public void setProfesorEncargado(String v) { profesorEncargado = v; }
    public int getCantidadEstudiantes() { return cantidadEstudiantes; }
    public void setCantidadEstudiantes(int v) { cantidadEstudiantes = v; }

    public void mostrarInformacion() {
        System.out.println("Curso: " + nombre + " (" + codigo + ")"
            + " - Profesor: " + profesorEncargado);
    }

    public void agregarEstudiante() {
        cantidadEstudiantes++;
    }
}

```

Clase Nota

```

public class Nota {
    private double valor;
    private String tipoEvaluacion;
    private String fecha;

    public Nota(double valor, String tipoEvaluacion, String fecha) {
        this.valor = valor;
        this.tipoEvaluacion = tipoEvaluacion;
        this.fecha = fecha;
    }

    public double getValor() { return valor; }
    public void setValor(double v) { valor = v; }
    public String getTipoEvaluacion() { return tipoEvaluacion; }
    public void setTipoEvaluacion(String v) { tipoEvaluacion = v; }
    public String getFecha() { return fecha; }
    public void setFecha(String v) { fecha = v; }

    public void mostrarNota() {
        System.out.println(tipoEvaluacion + " (" + fecha + "): " + valor);
    }

    public boolean esAprobatoria() {
        return valor >= 70;
    }
}

```

Clase Profesor

```

public class Profesor {
    private String identificacion;

```



```

private String nombre;
private String especialidad;

public Profesor(String identificacion, String nombre, String especialidad) {
    this.identificacion = identificacion;
    this.nombre = nombre;
    this.especialidad = especialidad;
}

public String getIdentificacion() { return identificacion; }
public void setIdentificacion(String v) { identificacion = v; }
public String getNombre() { return nombre; }
public void setNombre(String v) { nombre = v; }
public String getEspecialidad() { return especialidad; }
public void setEspecialidad(String v) { especialidad = v; }

public void mostrarDatos() {
    System.out.println("Profesor: " + nombre + " - Especialidad: " + especialidad);
}

public void asignarCurso(Curso c) {
    System.out.println(nombre + " fue asignado al curso " + c.getNombre());
}
}

```

Ojo con esto: en todos los constructores se inicializan los atributos que definen "quién es" el objeto (identificación, nombre, etc.). El atributo `promedio` de `Estudiante` y `cantidadEstudiantes` de `Curso` no se piden como parámetro porque tiene más sentido que empiecen en 0 y se actualicen después con su `set` o su método correspondiente, a medida que el estudiante va sacando notas.

5. El método main: qué pasa cuando se crea un objeto

Indicador 5

Todas las clases de arriba son solo "planos" o "moldes": no hacen nada por sí solas hasta que algo las use. Ese "algo" es el método `main`, que es el punto donde arranca cualquier programa en Java. Ahí es donde se crean los objetos con `new` y se les pide que hagan cosas llamando a sus métodos.

```
public class Main {
    public static void main(String[] args) {

        Profesor prof1 = new Profesor("204560789", "Marcela Solano", "Programación");
        Curso curso1 = new Curso("PRW-11B", "Programación Web", prof1.getNombre());
        Estudiante e1 = new Estudiante("118760123", "Damian Quirós", "damian@correo.com");

        curso1.agregarEstudiante();
        e1.setPromedio(88.5);
        Nota nota1 = new Nota(88.5, "Proyecto P00", "14/10/2026");

        prof1.mostrarDatos();
        curso1.mostrarInformacion();
        e1.mostrarDatos();
        nota1.mostrarNota();
    }
}
```

Con sus propias palabras, esto es lo que ocurre paso a paso desde que se crea un objeto hasta que se ejecuta un método sobre él, usando como ejemplo la línea `Estudiante e1 = new Estudiante("118760123", "Damian Quirós", "damian@correo.com");`:

- 1 **La JVM arranca el programa buscando `main`**. Es el único método que Java ejecuta automáticamente al iniciar; todo lo demás pasa porque `main` lo llama, directa o indirectamente.
- 2 **Java lee `new Estudiante(...)` y reserva memoria**. La palabra `new` le dice a Java "necesito espacio para guardar un objeto de tipo Estudiante". Ese espacio se reserva en una zona de memoria llamada *heap*, y ahí quedan guardados sus atributos (`identificacion`, `nombre`, `promedio`, `correo`), todavía sin valores definitivos.
- 3 **Se ejecuta el constructor**. Automáticamente corre el código de `public Estudiante(String identificacion, String nombre, String correo)`, que toma los tres textos que se le pasaron (`"118760123"`, `"Damian Quirós"`, `"damian@correo.com"`) y los guarda dentro de ese espacio de memoria que se acaba de reservar. El promedio se inicializa en 0.0.
- 4 **Se guarda la referencia en la variable `e1`**. La variable `e1` no contiene el objeto en sí; contiene la *dirección* de dónde quedó ese objeto en el heap. Es como una etiqueta con la que el resto del programa puede "encontrar" ese objeto en particular.
- 5 **El objeto ya existe y está listo para recibir mensajes**. A partir de aquí, cualquier línea que use `e1.algo()` va directo a buscar ese objeto específico en memoria y ejecuta el método pedido sobre sus datos, no sobre los de otro estudiante.

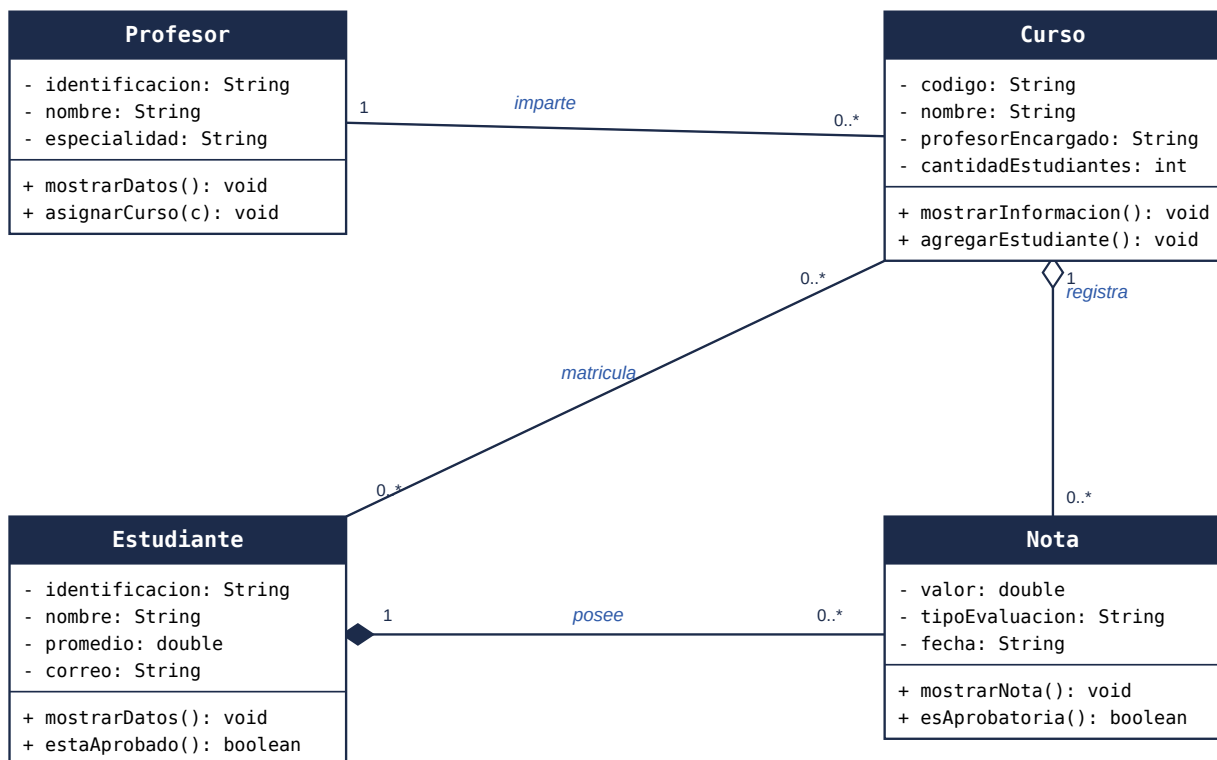
- 6 **Se ejecuta un método, por ejemplo `e1.mostrarDatos()`** . Java busca el objeto al que apunta `e1` , entra al método `mostrarDatos()` definido en la clase `Estudiante` , y ese método usa los atributos *de ese objeto* (su nombre, su identificación, su promedio) para imprimir el resultado en pantalla.
- 7 **Cuando `main` termina, el programa acaba.** Los objetos que ya no tiene sentido guardar quedan disponibles para que Java libere esa memoria más adelante de forma automática.

Idea clave: cada `new` crea un objeto *independiente*. Si en el código de arriba se creara un segundo estudiante `Estudiante e2 = new Estudiante(...)` , sería un objeto totalmente separado de `e1` , con su propia copia de `nombre` , `promedio` , etc., aunque los dos hayan salido del mismo molde (la misma clase).

6. Diagrama de clases (UML)

Indicador 6

Este diagrama resume en una sola imagen todo lo explicado en los puntos anteriores: las cuatro clases, sus atributos (con visibilidad `-` privada), sus métodos (con visibilidad `+` pública), y cómo se relacionan entre sí, con su multiplicidad.



Cómo leer el diagrama:

"-" atributo privado · "+" método público línea simple = asociación

◆ rombo relleno = composición (la Nota no existe sin el Estudiante)

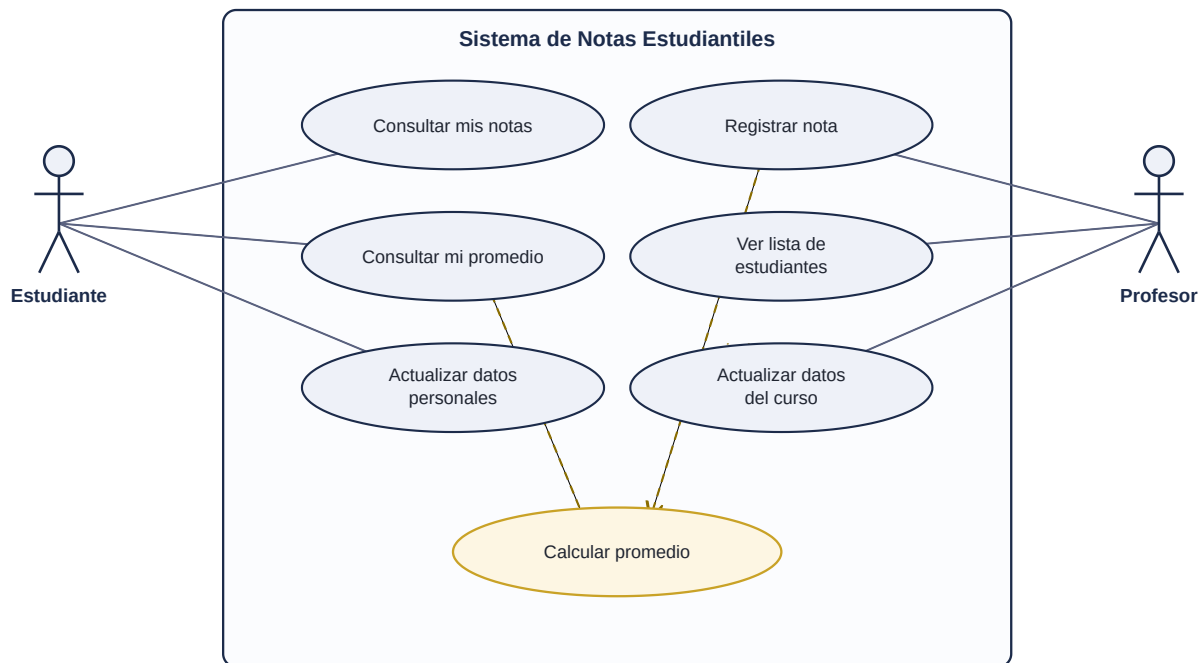
◇ rombo vacío = agregación (la Nota puede existir sin ese Curso específico)

"1", "0..*" = multiplicidad (de cero a muchos)

7. Diagrama de casos de uso (UML)

Indicador 7

Este diagrama muestra el sistema desde afuera: quiénes lo usan (los actores) y qué pueden hacer con él (los casos de uso), sin entrar en el detalle de cómo está programado por dentro.



Cómo leer el diagrama:

muñeco = actor (quién usa el sistema) óvalo = caso de uso (una función que el sistema ofrece)

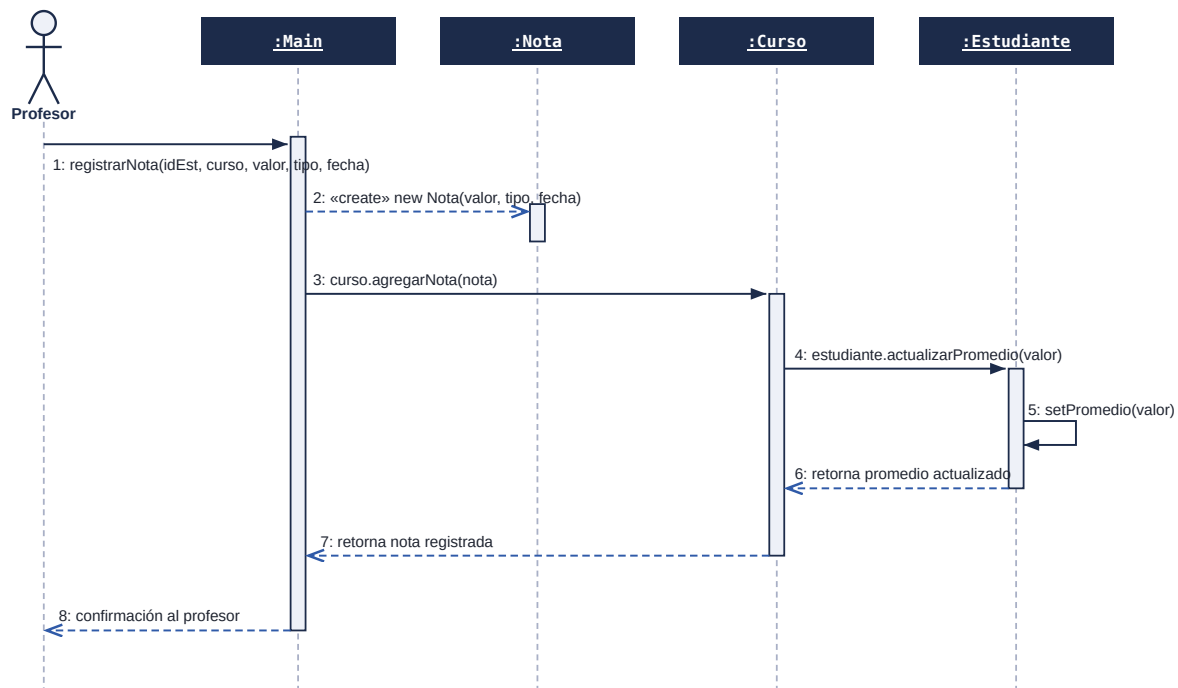
rectángulo grande = límite del sistema

flecha punteada «include» = ese caso de uso siempre necesita ejecutar también "Calcular promedio"

8. Diagrama de secuencia (UML)

Indicador 8

El diagrama de secuencia muestra, en orden de tiempo (de arriba hacia abajo), los mensajes que se intercambian los objetos en un caso concreto: **un profesor registra una nota nueva para un estudiante**, que es justamente el caso de uso "Registrar nota" del diagrama anterior.



Cómo leer el diagrama:

línea punteada vertical = línea de vida de cada objeto rectángulo angosto = activación (el objeto está procesando)

flecha sólida = llamada a un método flecha punteada = mensaje de retorno / respuesta

«create» = mensaje que crea un objeto nuevo con `new`

Conclusión

Con este documento queda diseñada, de forma teórica, la base del Sistema de Notas Estudiantiles: cuatro clases (`Estudiante` , `Curso` , `Nota` y `Profesor`) con sus atributos privados, sus métodos `get / set` , sus constructores, y la explicación de cómo el método `main` convierte esos "moldes" en objetos reales en memoria. Los tres diagramas UML (clases, casos de uso y secuencia) representan gráficamente esa misma estructura desde tres ángulos distintos: qué tiene cada clase por dentro, quién usa el sistema desde afuera, y cómo se comunican los objetos en un caso concreto.

Este diseño no se hizo pensando solo en esta etapa: las mismas cuatro clases son las que se van a usar en la Etapa 2 para trabajar bucles, arreglos y matrices (por ejemplo, un arreglo de `Estudiante` o una matriz de `Nota`), y en la Etapa 3 para agregar la superclase `Persona` , aplicar herencia y polimorfismo, y finalmente construir la interfaz gráfica en NetBeans. La idea es que las tres etapas se sientan como un solo proyecto que va creciendo, no como tres trabajos sueltos.